

Name: Kashinath Sangram Hale

PRN no.: 202501040055

Division: CS-1 Batch:

C13 Course: EDS

Essential of Data Science Laboratory

Practical 4:

4.1.1 Pandas - Series Creation and Manipulation:

CODETANTRA

Home

Learn Anywhere

202501040055@mitaoe.ac.inSupportLogout

4.1.1. Pandas - series creation and manipulation00:51

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases+

seriesMa...

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 series = pd.Series(numbers)
8 # Grouping by even and odd numbers and calculating the mean
9 grouped = series.groupby(series%2 == 0).mean()
10
11 # Display the mean of even and odd numbers with labels
12 grouped.index = ['Even' if is_even else 'Odd' for is_even in
13                 grouped.index]
14 print("Mean of even and odd numbers:")
15 print(grouped)
```

Average time0.021 s21.17 msMaximum time0.068 s68.00 ms

3 out of 3 shown test case(s) passed3 out of 3 hidden test case(s) passed

Test case 168 msDebug

Expected output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Odd5.0

Even6.0

dtype: float64

Actual output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Odd5.0

Even6.0

dtype: float64

TerminalTest cases

< Prev

Reset

Submit

Next >

4.1.2 Dictionary to dataframe:

CODETANTRA

Home

Learn Anywhere

202501040055@mitaoe.ac.inSupportLogout

4.1.2. Dictionary to dataframe38.33

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

datafram...

```
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
12 # Display the original DataFrame
13 print("Original DataFrame:")
14 print(df)
15
16 # Adding a new row
17 new_name = input("New name: ")
18 new_age = input("New age: ")
19 df.loc[len(df)] = [new_name, new_age]
20 df["Age"] = df["Age"].astype(int)
21 # Display the DataFrame after adding a new row
22 print("After adding a row:\n",df)
23
24 # Modifying a row
25 row_to_modify = int(input("Index of row to modify: "))
26 new_age_value = int(input("New age: "))
27 df.at[row_to_modify, "Age"] = new_age_value
28 # Display the DataFrame after modifying a row
29 print("After modifying a row:")
30 print(df)
31
```

TerminalTest cases

< Prev

Reset

Submit

Next >

CODETANTRA

Home

Learn Anywhere

202501040055@mitaoe.ac.inSupportLogout

4.1.2. Dictionary to dataframe38.33

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

datafram...

```
27 df.at[row_to_modify, "Age"] = new_age_value
28 # Display the DataFrame after modifying a row
29 print("After modifying a row:")
30 print(df)
31
32 # Deleting a row
33 row_to_delete = int(input("Index of row to delete: "))
34 df = df.drop(index=row_to_delete).reset_index(drop=True)
35 # Display the DataFrame after deleting a row
36 print("After deleting a row:")
37 print(df)
38
39 # Adding a new column
40 genders = input("Enter genders separated by space: ").split()
41 df["Gender"] = genders
42 # Display the DataFrame after adding a new column
43 print("After adding a new column:")
44 print(df)
45
46 # Modifying a column
47 df["Name"] = df["Name"].str.upper()
48 # Display the DataFrame after modifying a column
49 print("After modifying a column:")
50 print(df)
51
52 # Deleting a column
53 df = df.drop(columns=["Age"])
54 # Display the DataFrame after deleting a column
55 print("After deleting a column:")
56 print(df)
57
```

TerminalTest cases

< Prev

Reset

Submit

Next >

[Home](#)
[Learn Anywhere](#)

202501040055@mitaoe.ac.in
Support
Logout

Average time
0.220 s
220.00 ms

Maximum time
0.267 s
267.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1 267 ms
Debug

Expected output

Original DataFrame:

```

.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
New name: Susan
New age: 29
After adding a row:
.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
3....Susan...29
Index of row to modify: 0
New age: 27
After modifying a row:
.....Name..Age
0....Alice...27
1....Bob...30
2..Charlie...35
3....Susan...29
Index of row to delete: 3
After deleting a row:

```

Actual output

Original DataFrame:

```

.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
New name: Susan
New age: 29
After adding a row:
.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
3....Susan...29
Index of row to modify: 0
New age: 27
After modifying a row:
.....Name..Age
0....Alice...27
1....Bob...30
2..Charlie...35
3....Susan...29
Index of row to delete: 3
After deleting a row:

```

< Prev
Reset
Submit
Next >

4.1.3 Student Information:

[Home](#)
[Learn Anywhere](#)

202501040051@mitaoe.ac.in
Support
Logout

4.1.3. Student Information
17:43

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases

studentin...

Submit

```

1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
6 # write your code here..
7 print("First five rows:")
8 print(data.head())
9
10 average_age = round(data['Age'].mean(), 2)
11 print(f"Average age: {average_age}")
12
13 filter_students = data[data["Grade"]<="B"]
14 print("Students with a grade up to B")
15 print(filter_students)

```

Average time
0.050 s
49.60 ms

Maximum time
0.072 s
72.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1 72 ms
Debug

Expected output

student data.txt

```

First five rows:

```

Actual output

student data.txt

```

First five rows:

```

Terminal
Test cases

< Prev
Reset
Submit
Next >

4.2.1 Month with Highest total Sales:

CODETANTRA

Home

Learn Anywhere

202501040055@mitaoe.ac.inSupportLogout

4.2.1. Month with the Highest Total Sales10:12

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 # Find the month with the highest total sales
9 best_month = (df.iloc[:, 2] * df.iloc[:, 3]).groupby(pd.to_datetime(df.iloc[:, 0]).dt.strftime('%Y-%m')).sum().idxmax()
10 highest_sales = (df.iloc[:, 2] * df.iloc[:, 3]).groupby(pd.to_datetime(df.iloc[:, 0]).dt.strftime('%Y-%m')).sum().max()
11 print(f"Best month: {best_month}")
12 print(f"Total sales: ${highest_sales:.2f}")
```

Average time0.049 s49.00 msMaximum time0.094 s94.00 ms

1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 194 msDebug

Expected output	Actual output
sales_data.csv	sales_data.csv
Best month: 2025-01	Best month: 2025-01
Total sales: \$1210.00	Total sales: \$1210.00

TerminalTest cases

4.2.2 Best Selling Product:

CODETANTRA

Home

Learn Anywhere

202501040055@mitaoe.ac.inSupportLogout

4.2.2. Best Selling Product03:17

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 # Find the product with the highest total quantity sold
9 best_product = df.groupby('Product')['Quantity'].sum().idxmax()
10 highest_quantity = df.groupby('Product')['Quantity'].sum().max()
11
12 # Display the result
13 print(f"Best selling product: {best_product}")
14 print(f"Total quantity sold: {highest_quantity}")
15
```

Average time0.039 s39.00 msMaximum time0.079 s79.00 ms

1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 179 msDebug

Expected output	Actual output
sales_data.csv	sales_data.csv
Best selling product: Product A	Best selling product: Product A
Total quantity sold: 23	Total quantity sold: 23

TerminalTest cases

4.2.3 City that Sold Most Products:

CODETANTRA

Home Learn Anywhere

202501040055@mitaoe.ac.in Support Logout

4.2.3. City that Sold the Most Products01:56

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10 best_city = df.groupby('City')['Quantity'].sum().idxmax()
11
12 # Display the result
13 print(f"City sold the most products: {best_city}")
14
```

Average time0.059 s59.33 ms

Maximum time0.143 s143.00 ms

1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1143 ms

Expected output

Actual output

City sold the most products: Los Angeles

Terminal Test cases

< Prev Reset Submit Next >

4.2.4 Most Frequently Sold Product Pairs:

CODETANTRA

Home Learn Anywhere

202501040055@mitaoe.ac.in Support Logout

4.2.4. Most Frequently Sold Product Pairs01:43

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

frequen...

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 daily_products = df.groupby('Date')['Product'].apply(list)
12
13 # Initialize the counter to keep track of pair frequencies
14 pair_counts = Counter()
15
16 for products in daily_products:
17     # Sort and use set() to ensure we get unique, consistently ordered
18     # pairs (A, B is the same as B, A)
19     unique_products = sorted(set(products))
20     # Generate all pairs of size 2 from the day's products and add
21     # them to the counter
22     pair_counts.update(combinations(unique_products, 2))
23
24 # Output the most frequent product pairs
25 if pair_counts:
26     # Find the highest frequency count
27     max_count = max(pair_counts.values())
28     # Extract all pairs that match the highest frequency
29     best_pairs = [pair for pair, count in pair_counts.items() if count == max_count]
```

Terminal Test cases

< Prev Reset Submit Next >

CODETANTRA

HomeLearn Anywhere

202501040055@mitaoe.ac.inSupportLogout

4.2.4. Most Frequently Sold Product Pairs01:43

Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles

Explanation:

Transactions:

• 2025-01-01: Product A, Product B

• 2025-01-02: Product A, Product C

• 2025-01-03: Product B, Product A

• 2025-01-04: Product C, Product B

• 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

• Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.

• Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.

• Product B and Product C: Appear in transactions on 2025-01-04.

Most Frequent Product Combinations:

• Product A and Product B (2 times)

• Product A and Product C (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

frequen...

21 them to the counter

22 pair_counts.update(combinations(unique_products, 2))

23

24 # Output the most frequent product pairs

25 if pair_counts:

26 # Find the highest frequency count

27 max_count = max(pair_counts.values())

28

29 # Extract all pairs that match the highest frequency

30 best_pairs = [pair for pair, count in pair_counts.items() if count == max_count]

31

32 # Sort the results alphabetically to match the expected test case output

33 for pair in sorted(best_pairs):

34 print(f"{pair[0]} and {pair[1]}: {max_count} times")

Average time0.059 s59.33 ms

Maximum time0.132 s132.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1132 ms

Expected output

Actual output

sales_data.csv

Product A and Product B: 2 times

Product A and Product C: 2 times

Product A and Product B: 2 times

Product A and Product C: 2 times

Terminal

Test cases

< Prev

Reset

Submit

Next >

4.2.5 Titanic Dataset Analysis:

CODETANTRA

HomeLearn Anywhere

202501040055@mitaoe.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning05:56

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.

2. Display the last 5 rows of the dataset.

3. Get the shape of the dataset (number of rows and columns).

4. Get a summary of the dataset (using .info()).

5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().

6. Check for missing values and display the count of missing values for each column.

7. Fill missing values in the 'Age' column with the median age.

8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).

9. Drop the 'Cabin' column due to many missing values.

10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

titanicDat...

1 import pandas as pd

2 import numpy as np

3

4 # Load the Titanic dataset

5 data = pd.read_csv('Titanic-Dataset.csv')

6

7 # 1. Display the first 5 rows of the dataset

8 print(data.head())

9

10 # 2. Display the last 5 rows of the dataset

11 print(data.tail())

12

13 # 3. Get the shape of the dataset

14 print(data.shape)

15

16 # 4. Get a summary of the dataset (info)

17 print(data.info())

18

19 # 5. Get basic statistics of the dataset

20 print(data.describe())

21

22 # 6. Check for missing values

23 print(data.isnull().sum())

24

25 # 7. Fill missing values in the 'Age' column with the median age

26 data['Age'].fillna(data['Age'].median(), inplace=True)

27

28 # 8. Fill missing values in the 'Embarked' column with the mode

29 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

30

31 # 9. Drop the 'Cabin' column due to many missing values

Terminal

Test cases

< Prev

Reset

Submit

Next >

CODETANTRAHomeLearn Anywhere202501040055@mitaoe.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning08:56

9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

titanicDat...

22# 6. Check for missing values
23print(data.isnull().sum())
24
25# 7. Fill missing values in the 'Age' column with the median age
26data['Age'].fillna(data['Age'].median(), inplace=True)
27
28# 8. Fill missing values in the 'Embarked' column with the mode
29data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
30
31# 9. Drop the 'Cabin' column due to many missing values
32data.drop('Cabin', axis=1, inplace=True)
33
34# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
35data['FamilySize'] = data['SibSp'] + data['Parch']

Average time0.470 s
Maximum time0.470 s
1 out of 1 shown test case(s) passed

Test case 1470 ms
Expected output
Actual output
TerminalTest cases

4.2.6 Titanic Dataset Analysis:

5

CODETANTRAHomeLearn Anywhere202501040055@mitaoe.ac.inSupportLogout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 213:04

You are provided with the Titanic dataset containing information about passengers on the Titanic.
Your task is to write Python code to answer the following questions based on the dataset.
1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

titanicDat...

1import pandas as pd
2import numpy as np
3
4# Load the Titanic dataset
5data = pd.read_csv('Titanic-Dataset.csv')
6data['FamilySize'] = data['SibSp'] + data['Parch']
7
8# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
9# Assuming 'FamilySize' was created in the previous step.
10# Alternatively, you could use: data['IsAlone'] = ((data['SibSp'] +
11data['Parch']) == 0).astype(int)
12data['IsAlone'] = (data['FamilySize'] == 0).astype(int)
13
14# 2. Convert 'Sex' to numeric (male: 0, female: 1)
15data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
16
17# 3. One-hot encode the 'Embarked' column
18data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
19
20# 4. Get the mean age of passengers
21print(data['Age'].mean())
22
23# 5. Get the median fare of passengers
24print(data['Fare'].median())
25
26# 6. Get the number of passengers by class
27print(data['Pclass'].value_counts())
28
29# 7. Get the number of passengers by gender
30print(data['Sex'].value_counts())
31
32TerminalTest cases

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

13.04

- Get the number of passengers by survival status.
- Calculate the survival rate of passengers.
- Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achen)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

+

titanicDat...

```
27
28 # 7. Get the number of passengers by gender
29 print(data['Sex'].value_counts())
30
31 # 8. Get the number of passengers by survival status
32 print(data['Survived'].value_counts())
33
34 # 9. Calculate the survival rate
35 print(data['Survived'].mean())
36
37 # 10. Calculate the survival rate by gender
38 print(data.groupby('Sex')['Survived'].mean())
```

Average time

0.151 s

151.00 ms

Maximum time

0.151 s

151.00 ms

1 out of 1 shown test case(s) passed

Test case 1 151 ms

Debug

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3....491	3....491
1....216	1....216
2....184	2....184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0....577	0....577
1....314	1....314
Name: Sex, dtype: int64	Name: Sex, dtype: int64

Terminal Test cases

< Prev Reset Submit Next >

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

01.49

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- Calculate the survival rate by class.
- Calculate the survival rate by embarked location (Embarked_S).
- Calculate the survival rate by family size (FamilySize).
- Calculate the survival rate by being alone (IsAlone).
- Get the average fare by passenger class (Pclass).
- Get the average age by passenger class (Pclass).
- Get the average age by survival status (Survived).
- Get the average fare by survival status (Survived).
- Get the number of survivors by class (Pclass).
- Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,,Q
```

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10 # 1. Calculate the survival rate by class
11 print(data.groupby('Pclass')['Survived'].mean())
12
13 # 2. Calculate the survival rate by embarked location
14 # Note: pd.get_dummies with drop_first=True usually creates
15 # 'Embarked_Q' and 'Embarked_S'
16 print(data.groupby('Embarked_S')['Survived'].mean())
17
18 # 3. Calculate the survival rate by family size
19 print(data.groupby('FamilySize')['Survived'].mean())
20
21 # 4. Calculate the survival rate by being alone
22 print(data.groupby('IsAlone')['Survived'].mean())
23
24 # 5. Get the average fare by class
25 print(data.groupby('Pclass')['Fare'].mean())
26
27 # 6. Get the average age by class
28 print(data.groupby('Pclass')['Age'].mean())
29
30 # 7. Get the average age by survival status
31 print(data.groupby('Survived')['Age'].mean())
```

Terminal Test cases

< Prev Reset Submit Next >

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

01:49

- Get the average fare by survival status (Survived).
- Get the number of survivors by class (Pclass).
- Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achen)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases



titanicDat...

```
27 print(data.groupby('Pclass')['Age'].mean())
28
29 # 7. Get the average age by survival status
30 print(data.groupby('Survived')['Age'].mean())
31
32 # 8. Get the average fare by survival status
33 print(data.groupby('Survived')['Fare'].mean())
34
35 # 9. Get the number of survivors by class
36 print(data[data['Survived'] == 1]['Pclass'].value_counts())
37
38 # 10. Get the number of non-survivors by class
39 print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

Average time
0.172 s
172.00 ms

Maximum time
0.172 s
172.00 ms

1 out of 1 shown test case(s) passed

Test case 1 172 ms

Debug

Expected output

Actual output

Pclass

Pclass

1...0.629630

1...0.629630

2...0.472826

2...0.472826

3...0.242363

3...0.242363

Name: Survived, dtype: float64

Name: Survived, dtype: float64

Embarked_S

Embarked_S

0...0.506073

0...0.506073

1...0.336957

1...0.336957

Name: Survived, dtype: float64

Name: Survived, dtype: float64

Terminal

Test cases

Prev

Reset

Submit

Next

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

10:24

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- Get the number of survivors by gender (Sex).
- Get the number of non-survivors by gender (Sex).
- Get the number of survivors by embarkation location (Embarked_S).
- Get the number of non-survivors by embarkation location (Embarked_S).
- Calculate the percentage of children (Age < 18) who survived.
- Calculate the percentage of adults (Age >= 18) who survived.
- Get the median age of survivors.
- Get the median age of non-survivors.
- Get the median fare of survivors.
- Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,,Q
```

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9 # 1. Get the number of survivors by gender
10 print(data[data['Survived'] == 1]['Sex'].value_counts())
11
12 # 2. Get the number of non-survivors by gender
13 print(data[data['Survived'] == 0]['Sex'].value_counts())
14
15 # 3. Get the number of survivors by embarked location
16 print(data[data['Survived'] == 1]['Embarked_S'].value_counts())
17
18 # 4. Get the number of non-survivors by embarked location
19 print(data[data['Survived'] == 0]['Embarked_S'].value_counts())
20
21 # 5. Calculate the percentage of children (Age < 18) who survived
22 print(data[data['Age'] < 18]['Survived'].mean())
23
24 # 6. Calculate the percentage of adults (Age >= 18) who survived
25 print(data[data['Age'] >= 18]['Survived'].mean())
26
27 # 7. Get the median age of survivors
28 print(data[data['Survived'] == 1]['Age'].median())
29
30 # 8. Get the median age of non-survivors
31 print(data[data['Survived'] == 0]['Age'].median())
```

Terminal

Test cases

Prev

Reset

Submit

Next

CODETANTRAHomeLearn Anywhere

202501040055@mitaoe.ac.inSupportLogout

4.2.8. Titanic Dataset Analysis and Data Cleaning - 410:24

9. Get the median age of survivors.

10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

titanicDat...

Submit

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

print(data[data['Age'] >= 18]['Survived'].mean())

6. Calculate the percentage of adults (Age >= 18) who survived

print(data[data['Age'] >= 18]['Survived'].mean())

7. Get the median age of survivors

print(data[data['Survived'] == 1]['Age'].median())

8. Get the median age of non-survivors

print(data[data['Survived'] == 0]['Age'].median())

9. Get the median fare of survivors

print(data[data['Survived'] == 1]['Fare'].median())

10. Get the median fare of non-survivors

print(data[data['Survived'] == 0]['Fare'].median())

Average time0.188 s188.00 ms

Maximum time0.188 s188.00 ms

1 out of 1 shown test case(s) passed

Test case 1188 ms

Debug

Expected output

Actual output

female...233

female...233

male...109

male...109

Name: Sex, dtype: int64

Name: Sex, dtype: int64

male...468

male...468

female...81

female...81

Name: Sex, dtype: int64

Name: Sex, dtype: int64

Terminal

Test cases

Prev

Reset

Submit

Next